

Java Backend Architecture

Interview Series

Chapter 19.4

AWS Storage & Messaging

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.4 – AWS Storage & Messaging

Overview

S3, SQS, and SNS are the foundational AWS services for Java backend storage and asynchronous messaging. S3 provides durable object storage; SQS enables decoupled queue-based messaging; SNS provides pub/sub fan-out. Understanding presigned URLs, FIFO vs standard queues, dead-letter queues, and SNS message filtering is essential for Java microservice architects.

Key Definitions

Term	Definition
S3 Bucket	Object storage. 11 nines durability. Globally unique name, region-specific data residency.
Presigned URL	Time-limited S3 URL granting access without AWS credentials. Issued by Java backend.
S3 Lifecycle Rules	Automate object transition (S3 -> IA -> Glacier) or expiration based on age or prefix.
Multipart Upload	Upload large files (>100MB) in parts concurrently. Required for >5GB; recommended >100MB.
SQS Standard Queue	At-least-once delivery, best-effort ordering. Up to 120k in-flight messages.
SQS FIFO Queue	Exactly-once delivery within message group, strict ordering. 300 TPS (3000 with batching).
Visibility Timeout	Duration a received SQS message is hidden from other consumers while being processed.
Dead-Letter Queue (DLQ)	SQS queue for messages that fail processing after maxReceiveCount attempts.
SNS Topic	Pub/sub messaging. Publishers send to topic; subscribers (SQS, Lambda, HTTP, Email) receive.
SNS Message Filtering	Subscribers receive only messages matching filter policy attributes. Reduces SQS/Lambda invocations.

Diagram 1: S3 Presigned URL Flow for Java

```

Java Backend (Spring Boot)
|
| 1. Client requests upload URL
|   GET /api/upload-url?filename=report.pdf
v
Java Service
|
| 2. Generate presigned URL (15 min TTL)
|   s3Client.generatePresignedUrl(bucket, key, expiration, HttpMethod.PUT)
v
Client receives: https://bucket.s3.amazonaws.com/report.pdf?X-Amz-Signature=...&Expires=...
|
| 3. Client uploads directly to S3 (bypasses Java backend)
v
S3 Bucket
|
| 4. S3 Event Notification -> SQS -> Java processor

```

Diagram 2: SQS/SNS Fan-out Pattern

```

Java Order Service
|
| publish: OrderPlaced{orderId, items}
v
SNS Topic: order-events
|
+-----+-----+
v         v         v
SQS:      SQS:      SQS:
email invoice warehouse
queue queue  queue
|         |         |
v         v         v
Email Invoice Inventory
Lambda Service Service
(fan-out: each subscriber gets every message)

```

Code Examples

S3 Operations in Java (AWS SDK v2):

```
// Upload with multipart (Spring Boot)
S3Client s3 = S3Client.builder().region(Region.US_EAST_1).build();

// Simple upload
s3.putObject(PutObjectRequest.builder()
    .bucket("java-app-bucket")
    .key("uploads/" + filename)
    .contentType("application/pdf")
    .serverSideEncryption(ServerSideEncryption.AES256)
    .build(), RequestBody.fromFile(file));

// Generate presigned URL (15 min)
S3Presigner presigner = S3Presigner.create();
PresignedPutObjectRequest presignedRequest = presigner.presignPutObject(r -> r
    .signatureDuration(Duration.ofMinutes(15))
    .putObjectRequest(p -> p.bucket(bucket).key(key)));
return presignedRequest.url().toString();
```

SQS Producer/Consumer in Java:

```
// Producer (Spring Boot)
@Service
public class OrderEventPublisher {
    private final SqsClient sqsClient;
    private final String queueUrl;

    public void publishOrder(Order order) {
        sqsClient.sendMessage(SendMessageRequest.builder()
            .queueUrl(queueUrl)
            .messageBody(objectMapper.writeValueAsString(order))
            .messageGroupId(order.getCustomerId()) // FIFO: per-customer ordering
            .messageDeduplicationId(order.getId()) // FIFO: idempotency
            .build());
    }
}

// Consumer (Spring Cloud AWS SQS Listener)
@SqsListener(value = "${order.queue.url}", maxConcurrentMessages = "10")
public void processOrder(Order order, Acknowledgement ack) {
    orderService.process(order);
    ack.acknowledge(); // explicit ack: message deleted from SQS
}
```

S3 Lifecycle Policy (Terraform):

```
resource "aws_s3_bucket_lifecycle_configuration" "java_logs" {
    bucket = aws_s3_bucket.java_logs.id

    rule {
        id = "log-retention"
        status = "Enabled"
        transition {
            days = 30
            storage_class = "STANDARD_IA" # cheaper for infrequent access
        }
        transition {
            days = 90
            storage_class = "GLACIER"
        }
        expiration { days = 365 } # delete after 1 year
    }
}
```

Interview Q&A – AWS Storage & Messaging

Q1. What is the difference between SQS Standard and FIFO queues?

Standard: at-least-once delivery (occasional duplicates), best-effort ordering, nearly unlimited throughput. Use for: email notifications, log processing, tasks where duplicates are handled idempotently. FIFO: exactly-once processing (message deduplication ID), strict per-group ordering, 300 TPS (3000 with batching). Use for: financial transactions, order processing where order matters and duplicates must be prevented.

Q2. How does SQS visibility timeout prevent duplicate processing?

When a Java consumer receives a message, SQS hides it for visibility_timeout seconds (default 30s). If consumer processes and deletes before timeout: message gone. If consumer crashes before deleting: timeout expires, message reappears for another consumer. Set visibility timeout > max processing time. Extend timeout if processing takes longer: `ChangeMessageVisibility`. maxReceiveCount + DLQ: after N failures, message moves to DLQ for inspection.

Q3. When should you use presigned S3 URLs vs server-side upload?

Presigned URL: client uploads directly to S3 -- bypasses Java backend, no memory pressure on server, no bandwidth cost on server, scales independently. Best for: large files (images, PDFs, videos), mobile uploads, batch file transfers. Server-side upload: Java receives file, validates/transforms, uploads to S3. Best for: files requiring server-side validation, format conversion, virus scanning before storage.

Q4. What is an SQS Dead-Letter Queue and when should you inspect it?

DLQ receives messages after maxReceiveCount failed processing attempts. Indicates: message format error (Java deserialization failure), unhandled exception in consumer, downstream service unavailable. Alert when DLQ message count > 0. Inspect DLQ to debug: read message, identify error pattern, fix consumer code, then redrive messages back to main queue using SQS Console or `StartMessageMoveTask` API.

Q5. How does SNS message filtering reduce downstream cost?

Without filtering: all SQS subscribers receive all SNS messages -- each Lambda/SQS invocation costs money even for irrelevant messages. SNS filter policy on subscription: `{eventType: [OrderPlaced, OrderCancelled]}` -- subscriber only receives matching messages. Java producer adds message attributes: `MessageAttribute eventType=OrderPlaced`. Reduces Lambda invocations and SQS receives for downstream Java services by 90%+ in high-volume event-driven architectures.

Q6. How do S3 event notifications work with Java backends?

S3 notifies on object creation/deletion/restore: send to SQS, SNS, or Lambda. S3 Event Notification -> SQS queue -> Java consumer (Spring `@SqsListener`). Event contains: bucket name, object key, size, event type. Java processes: generate thumbnail (image upload), extract text (PDF upload), validate file (antivirus). Use SQS as buffer (not direct Lambda trigger) for reliable processing and backpressure management.

Q7. What is S3 Transfer Acceleration and when to use it?

Transfer Acceleration uses CloudFront edge network to route uploads to S3 over AWS backbone (faster than public internet for long-distance uploads). Enable on bucket: `aws_s3_bucket_accelerate_configuration`. Use when: global users uploading to a single S3 region, large file uploads from distant locations. Not needed for same-region uploads. Cost: \$0.04/GB for accelerated upload (vs \$0.005/GB standard). Test with AWS's speed comparison tool.

Q8. How do you implement S3 server-side encryption for Java uploads?

SSE-S3: AWS-managed keys (AES-256). Simple, no cost. Set via bucket policy or `putObject` header (`x-amz-server-side-encryption: AES256`). SSE-KMS: AWS KMS-managed keys, key rotation, audit trail in CloudTrail. SSE-C: customer-provided keys (Java app manages keys, risky). Best for Java apps: SSE-KMS with `aws:kms` managed key -- minimal code change, compliance-friendly, key rotation automated. Enforce via bucket policy denying unencrypted puts.

Q9. How does SQS long polling reduce cost?

Short polling: SQS checks a subset of servers, may return empty even if messages exist. Java consumer polls repeatedly -- unnecessary API calls. Long polling (`WaitTimeSeconds=20`): SQS waits up to 20s for a message before returning empty response. Reduces empty receive calls by 95%+, reducing cost and CPU. Configure: `receive_message_wait_time_seconds` on queue (default) or per-request `WaitTimeSeconds`. Spring Cloud AWS: enabled by default.

Q10. What is S3 Versioning and when should Java apps use it?

Versioning maintains all versions of an object -- accidental deletion/overwrite recoverable. Enable: `aws s3api put-bucket-versioning --versioning-configuration Status=Enabled`. Versioned delete: adds delete marker, original preserved. Recover: delete the delete

marker. Use for: user-uploaded documents where auditing/recovery required, configuration files deployed by Java CI/CD (rollback to previous config). Pair with lifecycle rules to expire old versions after N days to control storage costs.